CI/CD The Kubernetes Way

Una piattaforma sovrana e multi-tenant con Tekton

Marco Russo · ReeVo Cloud & Cyber Security

`#PlatformEngineering` · `#DigitalSovereignty` · `#SupplyChainSecurity`

REEVO

REEVO

Chi sono

- **Marco Russo** · DevOps Engineer @ ReeVo Cloud & Cyber Security
- Piattaforme di rilascio su Kubernetes: golden path, sicurezza della supply chain, team in self-service
- ReeVo è un cloud provider europeo

LinkedIn

GitHub

REEVO

Il percorso di oggi

Chi controlla davvero la piattaforma di CI/CD di un'organizzazione?

1. **WHY** · perché la pipeline è diventata un bersaglio e chi la controlla oggi
2. **WHAT** · i requisiti di una piattaforma sicura e indipendente
3. **HOW** · come Kubernetes e Tekton coprono questi requisiti
4. **GOVERN** · template firmati e controlli che impediscono di aggirarli
5. **DEMO** · il contratto di piattaforma alla prova

1 · WHY

Perché il modello attuale non basta?

il bersaglio si sposta → la definizione non è portabile → la giurisdizione non si sposta

WHY WHAT HOW GOVERN

REEVO

REEVO

Il bersaglio si è spostato



2020	2021	2025	2025	2026
SolarWinds	**Codecov**	**tj-actions**	**Shai-Hulud**	**Trivy**
ambiente di build compromesso	secret delle pipeline esfiltrati	23.000 repo, secret nei log	worm auto-replicante su npm	scanner → credential stealer

Negli attacchi più recenti l'obiettivo è la piattaforma che costruisce il software, più che il software stesso.

Nel DBIR 2026 il coinvolgimento di terze parti tocca il **48% dei breach**, in crescita del 60% sull'edizione precedente.

Fonti: CISA (tj-actions, npm/Shai-Hulud); CVE-2025-30066; CVE-2026-33634; Verizon DBIR 2026

REEVO

Perché proprio la pipeline?

La pipeline ha accesso a tutto ciò che serve per produrre e distribuire il software:

- legge il codice sorgente, compreso quello privato;
- usa secret e credenziali cloud con permessi di scrittura;
- costruisce gli artefatti, li firma e spesso li pubblica in produzione.

Ogni tool aggiunto alla pipeline (scanner, linter, action di terze parti) può usare gli accessi concessi al suo step. Nel caso Trivy è stato proprio uno scanner a sottrarre le credenziali.

Una pipeline compromessa può distribuire artefatti alterati con firme valide: la firma da sola non garantisce che il software sia sicuro.

REEVO

Il lock-in è doppio

Un runner self-hosted sposta la build, ma non cambia il formato della pipeline né, da solo, chi custodisce i dati. Il lock-in riguarda due aspetti:

- la **definizione**: quanto costa riscrivere le pipeline su un'altra piattaforma;
- la **giurisdizione**: a quale ordinamento risponde chi custodisce sorgente, secret e artefatti.

La piattaforma di delivery è un asset critico: dobbiamo poterla governare e migrare.

REEVO

Lock-in 1 · La definizione non è portabile

Il costo della migrazione è ricostruire il contratto della CI, oltre a tradurre lo YAML:

- **Step riusabili**: `uses: owner/action@v4` su GitHub; `include:` e CI/CD Components su GitLab. Cambiano packaging e interfacce.
- **Espressioni e condizioni**: `${{ github.event... }}` e `strategy.matrix` contro `rules:` e `!reference`. La traduzione non è 1:1.
- **Integrazioni**: identità OIDC, approvazioni degli environment e report nella UI del vendor.

La doc di GitLab sulla migrazione da GitHub Actions lo dice esplicitamente: «not exact matches»

Lock-in 2 · La giurisdizione non si sposta

La residency indica dove sono conservati i dati. La sovranità riguarda invece chi ha autorità legale sul fornitore che li custodisce.

- **CLOUD Act**: un provider soggetto alla giurisdizione USA può essere obbligato a consegnare i dati che detiene, anche se sono conservati in UE.
- **Trump v. Slaughter** (2026): la Corte Suprema supera *Humphrey's Executor* ed estende il potere presidenziale di rimozione sulle agenzie indipendenti. Il possibile impatto sulle garanzie del Data Privacy Framework va distinto da una decisione sulla sua validità.

Sovranità: scegliere chi custodisce dati e chiavi, chi può accedervi e come cambiare fornitore.

Fonti: U.S. DOJ, CLOUD Act; U.S. Supreme Court, Trump v. Slaughter (2026)

2 · WHAT

Quali garanzie servono a una piattaforma sicura e indipendente?

provenance non falsificabile → giurisdizione interna → standard non aggirabili

WHY | **WHAT** | HOW | GOVERN

REEVO

Tre requisiti

R1 · Provenance non falsificabile

Ogni artefatto ha una provenance che gli step della build non possono modificare.

R2 · Sovranità dei dati

Sorgenti, run, chiavi di firma e attestazioni sono sotto il nostro controllo operativo e giuridico.

R3 · Standard non aggirabili

I team rilasciano in self-service entro template e confini imposti dalla piattaforma.

REEVO

R1 · I limiti della SBOM

La **SBOM** descrive il contenuto di un artefatto:

> **"Cosa c'è dentro questo artefatto?"**

Elenca componenti, versioni, dipendenze e licenze. Il **CRA** richiede una SBOM che copra almeno le dipendenze di primo livello.

La SBOM, da sola, non dimostra:

- chi l'ha generata, da quale sorgente e con quali dipendenze risolte;
- se proviene dalla stessa build che ha prodotto l'artefatto;
- se è stata modificata in seguito.

Se la build è compromessa, anche la SBOM può essere falsificata.

Fonti: CycloneDX, SPDX; Reg. UE 2024/2847 (CRA)

R1 · La provenance e i livelli SLSA

La **provenance** descrive come è stato prodotto un artefatto:

> **"Come, da dove e da chi è stato prodotto?"**

L'affidabilità della provenance dipende da chi la genera e la firma. **SLSA** la classifica con i livelli del **Build Track**:

- **L1**: la provenance esiste;
- **L2**: una build platform ospitata genera provenance autentica;
- **L3**: isolamento fra build e provenance resistente alla falsificazione da parte dei tenant. Chiave e generazione dei dati restano nel confine fidato.

3 · HOW

Come si costruisce la piattaforma?

Kubernetes → Task, Pipeline, PipelineRun → Pipelines-as-Code → Chains

WHY WHAT **HOW** GOVERN

REEVO

REEVO

Perché Kubernetes

Kubernetes offre i meccanismi su cui costruire i tre requisiti:

Requisito	Cosa offre Kubernetes
R1 · Provenance non falsificabile	Pod effimeri per TaskRun; controller di firma separato e accesso alle chiavi limitato alla piattaforma
R2 · Sovranità dei dati	Le stesse API on-prem o su cloud scelto da noi; controllo su custodia di dati, chiavi e attestazioni
R3 · Standard non aggirabili	Policy di ammissione che decidono quali risorse il cluster accetta di creare

Configurando l'audit delle API, le operazioni sui tenant hanno un formato comune. I log di build e i controlli di isolamento completano questa vista.

Tekton

Tekton è una soluzione **open-source** e **cloud-native** per la CI/CD su Kubernetes, composta da diversi moduli:

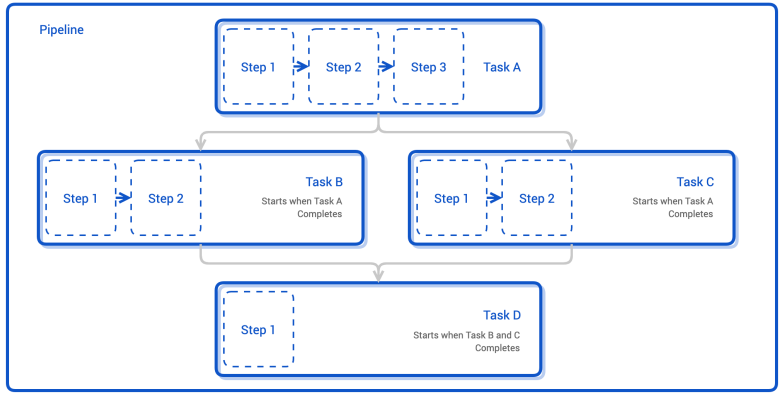
Componente	Cosa fa
Pipelines	Definisce ed esegue le pipeline
Chains	Firma artefatti e provenance
Pipelines-as-Code	Gestisce le pipeline dal repository
Triggers	Avvia le pipeline tramite webhook
Dashboard, Results	Mostra esecuzioni e risultati

Una forge self-hosted può dare controllo sui dati. **Tekton separa il motore di build dalla forge**: eventi e autenticazione possono cambiare, mentre Task e Pipeline restano sulla nostra piattaforma.

Fonti: tekton.dev; CNCF (Tekton incubation, mar. 2026); tektoncd/community, adopters.md

Il Task: l'unità riusabile

Un **Task** è una sequenza ordinata di **step**: ogni step invoca uno strumento dentro un container.



- Gli **step** di un Task girano nello stesso Pod e condividono rete e volumi montati. Il runtime fornisce il confine di esecuzione; RBAC governa l'accesso alle API.
- I **params** (input) e i **results** (output) sono dichiarati nel Task e ne definiscono l'interfaccia.

Come si scrive un Task

REEVO

```yaml
# Estratto: configurazione del builder e sorgenti omessi
apiVersion: tekton.dev/v1
kind: Task
metadata: { name: build-and-push }
spec:
  params: [{ name: IMAGE }]                # input: l'interfaccia verso il team
  workspaces: [{ name: source }]
  results: [{ name: BUILT_DIGEST }]        # output: lo legge il task successivo
  steps:
    - name: build
      image: docker.io/moby/buildkit@sha256:<digest>
      workingDir: $(workspaces.source.path)
      script: buildctl build --output type=oci,dest=image.tar,name=app:build
    - name: push
      image: ghcr.io/oras-project/oras@sha256:<digest>
      workingDir: $(workspaces.source.path)
      script: |
        oras cp --from-oci-layout image.tar:build $(params.IMAGE)
        oras resolve $(params.IMAGE) > $(results.BUILT_DIGEST.path)
```

Pipeline

Una **Pipeline** definisce quali `Task` eseguire e in quale ordine. Task senza dipendenze possono essere eseguiti in parallelo; `runAfter` e riferimenti ai risultati stabiliscono l'ordine.

- I **risultati** possono passare da un `Task` all'altro, per esempio con `$(tasks.build.results.BUILT_DIGEST)`.
- I **Task non condividono il filesystem**: ogni Task gira in un Pod distinto. Se devono scambiarsi file, si usa un **workspace**, che viene dichiarato dal `Task`, collegato dalla `Pipeline` e infine fornito dalla `PipelineRun` tramite un PVC. `emptyDir` è locale al Pod e non scambia file fra Task distinti.

Fonte: tekton.dev/docs/concepts/overview

REEVO

Come si scrive una pipeline

```yaml
apiVersion: tekton.dev/v1
kind: Pipeline
metadata: { name: release }
spec:
  params: [{ name: IMAGE }]
  tasks:
    - name: build                          # primo task
      taskRef: { name: build-and-push }
      params: [{ name: IMAGE, value: $(params.IMAGE) }]
    - name: scan                           # dopo build: ne legge il result
      runAfter: [build]
      taskRef: { name: trivy-scan }
      params: [{ name: DIGEST, value: $(tasks.build.results.BUILT_DIGEST) }]
```

Come si scrive una PipelineRun

REEVO

Una **PipelineRun** è una singola esecuzione di una Pipeline: indica quale Pipeline eseguire, con quali parametri e con quale identità.

```yaml
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata: { name: release-42 }
spec:
  pipelineRef: { name: release }            # cosa eseguire
  params: [{ name: IMAGE, value: registry.internal/team-a/api }]
  taskRunTemplate:
    serviceAccountName: team-a              # l'identità con cui esegue
```

- `pipelineRef` indica la Pipeline per nome nel cluster, oppure tramite un **resolver** che la recupera da Git o da un bundle OCI.
- I risultati dichiarati e mappati dalla Pipeline compaiono in `PipelineRun.status.results`; i type hint indicano a Chains quali artefatti firmare.

REEVO

Pipelines-as-Code

Le PipelineRun non vengono create a mano. Con **Pipelines-as-Code** il team le definisce nella cartella `.tekton/` del repository, e a ogni evento Git il cluster della piattaforma le crea e le esegue.

- La risorsa `Repository` collega un repository Git al namespace del team.
- Le run partono su eventi di PR e push; filtri e concorrenza si configurano con annotation.
- Dai commenti della PR si possono lanciare comandi (`/retest`, `/ok-to-test`), e lo stato della run viene riportato sulla PR.
- I provider supportati sono GitHub, GitLab, Bitbucket (Cloud e Data Center) e Forgejo.

Come si scrive con Pipelines-as-Code

REEVO

```yaml
# .tekton/pull-request.yaml — nel repository del team, accanto a .tekton/pipeline.yaml
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  name: pr-check
  annotations:
    pipelinesascode.tekton.dev/on-event: "[pull_request]"     # quando parte
    pipelinesascode.tekton.dev/on-target-branch: "[main]"
spec:
  pipelineRef: { name: release }       # risolta da .tekton/pipeline.yaml
  params:
    - { name: IMAGE, value: "registry.internal/team-a/api:{{ revision }}" }
```

- Le annotation indicano su quali eventi e branch parte la run; `{{ revision }}` viene sostituito con il commit dell'evento.
- Il team può scrivere qualsiasi `Pipeline` in `.tekton/`. La parte 4 introduce template condivisi e controlli per governare questa libertà.

Fonte: pipelinesascode.com, Authoring PipelineRuns

REEVO

Chains: la provenance firmata

Chains è un controller separato che osserva `TaskRun` e `PipelineRun`. Quando una run termina, raccoglie i risultati e costruisce la relativa provenance.

La provenance viene poi:

- formattata secondo lo standard configurato (`in-toto`/SLSA);
- firmata con il metodo scelto (`x509`, `kms` o nessuna firma);
- salvata nei backend configurati, ad esempio nelle annotazioni Tekton o nel registry OCI.

Per le immagini, Chains usa i parametri e i risultati del run per capire quali artefatti firmare. Le informazioni Git, invece, vengono usate per descrivere la sorgente nella provenance e non per decidere quali immagini firmare.

Fonte: tekton.dev, Chains — SLSA provenance

Il flusso di Chains

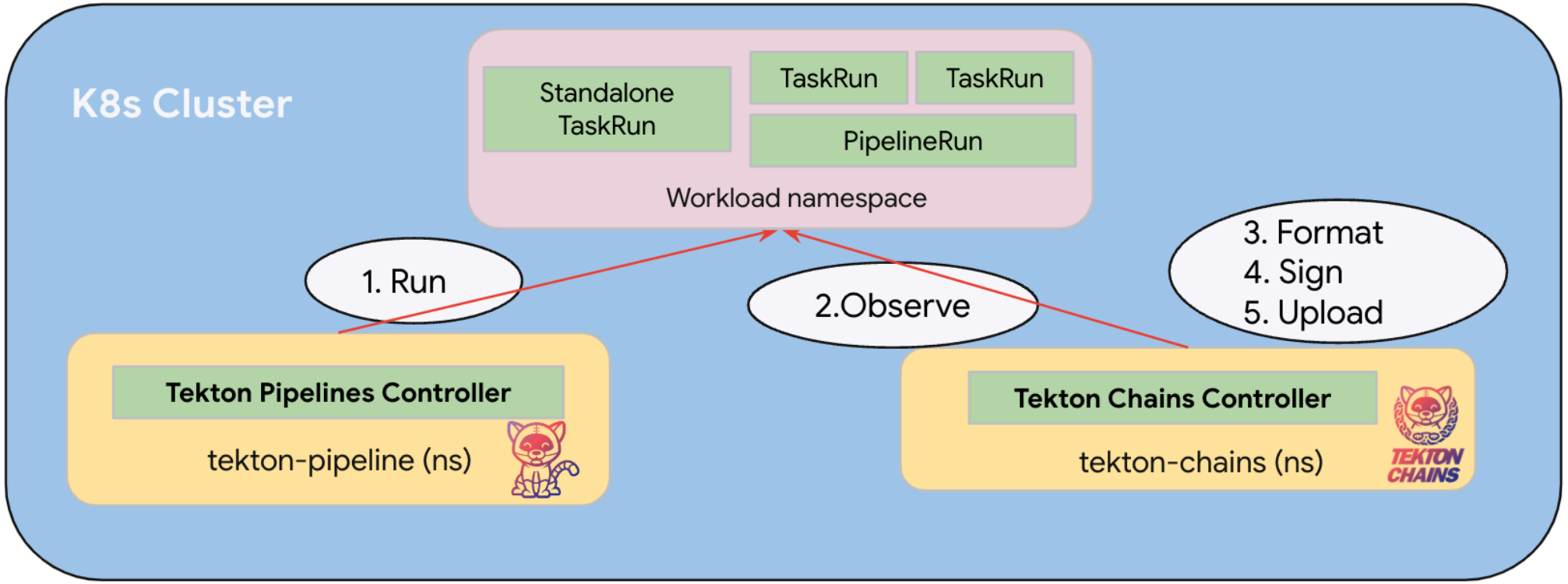


Diagramma: tekton.dev, Chains — how Chains works

4 · GOVERN

Come si impedisce che la piattaforma venga aggirata?

il cluster contiene → la piattaforma pubblica → il team referenzia → il cluster verifica

WHY WHAT HOW **GOVERN**

REEVO

REEVO

L'hardening del cluster è l'hardening della build

I Task diventano Pod: possiamo applicare alle build i controlli Kubernetes già usati per i tenant, scegliendo un profilo compatibile con il builder.

Meccanismo Kubernetes	Cosa protegge nella build
`namespace` + RBAC	Accesso alle API e ai secret limitato al team; il registry richiede ACL proprie
`ResourceQuota`, `LimitRange`	Limiti al consumo dei workload e capacità da dimensionare fra tenant
`NetworkPolicy` default-deny	Riduce i servizi e le destinazioni raggiungibili; richiede un CNI compatibile
`PodSecurityAdmission`	Niente privilegi, niente mount dall'host
`RuntimeClass` (gVisor, Kata)	Rafforza il confine di esecuzione, secondo il runtime scelto

Questi controlli limitano cosa può fare una build. Le slide seguenti mostrano come la piattaforma decide quale build può partire.

Il builder della demo usa Unconfined e non passa il profilo restricted

REEVO

Il bundle: cosa pubblica la piattaforma

Nella parte 3 ogni team scriveva la propria `Pipeline` nel repository. Per decidere quale build può partire, la piattaforma sposta questa definizione fuori dai repository e la pubblica come **bundle**.

Un bundle è un'immagine OCI che contiene una o più definizioni di `Task` e `Pipeline`. La piattaforma lo pubblica nel registry interno, accanto alle immagini e alle attestazioni, a ogni nuova versione dei template. Quando parte una run, il cluster lo scarica con il resolver `bundles`.

Ogni bundle è identificato da un digest: i team che scelgono quella versione eseguono la stessa definizione; una correzione si pubblica una volta sola, e il cluster può verificare che la pipeline in esecuzione sia quella approvata.

Fonte: tekton.dev, Bundles resolver

REEVO

Come la piattaforma pubblica un bundle

La pubblicazione della golden path avviene in tre passi:

1. **impacchetta** la Pipeline con i Task incorporati e pubblica il bundle OCI nel registry interno;
2. **firma e verifica** il bundle: Chains firma l'artefatto pubblicato, il publisher verifica la firma con `cosign`;
3. **approva** il digest verificato, aggiungendolo all'elenco accettato dalla policy di ammissione.

La firma prova l'autorità; il digest identifica il contenuto approvato.

Fonti: demo/scripts/publish.sh; Tekton Chains; Trusted Resources

La golden path: cosa scrive il team

Il team scrive solo `.tekton/release.yaml` e sceglie parametri e momento del rilascio. A differenza della parte 3, la `Pipeline` non è nel repository: arriva dal bundle, per digest.

```yaml
# Estratto: digest abbreviato; workspace e identità omessi
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  name: release
  annotations: { pipelinesascode.tekton.dev/on-event: "[push]" }   # quando parte
spec:
  pipelineRef:
    resolver: bundles                    # risolto dal registry interno
    params:
      - { name: bundle, value: registry.internal/tekton/pipelines@sha256:3b1f }
      - { name: name, value: release }
      - { name: kind, value: Pipeline }
  params: [{ name: IMAGE, value: registry.internal/team-a/api }]   # i suoi valori
```

I punti di controllo

La firma viene verificata prima dell'approvazione; l'ammissione impone il digest:

Quando	Chi verifica	Cosa controlla
pubblicazione	publisher (`cosign`)	firma OCI del bundle, prima di approvarne il digest
creazione della run	`ValidatingAdmissionPolicy`	digest approvato sul percorso `bundles`
risoluzione, opzionale	Tekton (`VerificationPolicy`, alpha)	firma in-resource; percorso separato dalla demo
deploy, nel blueprint	policy del cluster di destinazione	firma e contenuto atteso della provenance

Per R3, coprire anche definizioni inline, altri resolver e TaskRun dirette.

Fonti: Tekton, Trusted Resources; Kubernetes, ValidatingAdmissionPolicy; demo/platform/policies

Il gate: quando dichiarare l'artefatto

build/push → scan → SBOM → dichiarazione a Chains

```
# Estratto del task promote della golden path
when:
  - input: $(tasks.scan.results.SCAN_GATE)
    operator: in
    values: ["passed"]
```

Solo `promote` produce i type hint `IMAGE_URL` e `IMAGE_DIGEST` per la firma.

Scan fallito: l'immagine resta nel registry, ma non viene dichiarata a Chains.

REEVO

I limiti che restano

- **Contenuto dichiarato**: una firma valida non rende innocuo il codice del team. Vanno verificati anche sorgente, builder e parametri della provenance.
- **Isolamento tra team**: namespace e RBAC governano l'API; rete, registry e controller condivisi richiedono confini espliciti.
- **Isolamento delle build**: Pod effimeri e chiave separata sono parte del percorso verso L3, non una dimostrazione completa del livello.

Fonti: SLSA v1.2, Build requirements; BuildKit, rootless mode

Il blueprint

Ruolo	La scelta
Git provider	Forgejo nel cluster (o GitLab self-hosted)
Registry OCI	zot: immagini, **bundle firmati** e attestazioni
Motore CI/CD	Tekton Pipelines + Pipelines-as-Code
Firma	Tekton Chains; custodia, rotazione e audit delle chiavi, eventualmente in un **KMS interno**
Verifica	firma verificata prima dell'approvazione + digest in ammissione + policy al deploy; `VerificationPolicy` come controllo aggiuntivo
Isolamento e quote	un `namespace` per team, RBAC, `ResourceQuota`, sandbox di runtime (gVisor/Kata), quote e code con **Kueue**

La demo usa un Secret per Chains e verifica da CLI. Il blueprint aggiunge isolamento, gestione delle chiavi e verifica al deploy.

Il trade-off: possedere la piattaforma significa operarne controller, storage, chiavi e aggiornamenti.

Demo

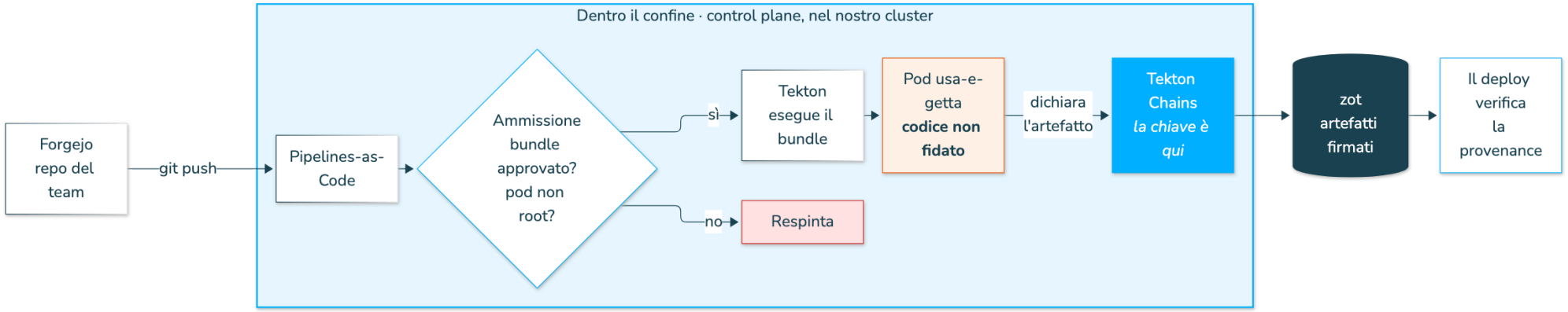
Quattro esempi sullo stesso cluster

1. Con un bundle **firmato** la run viene eseguita e l'immagine viene firmata con la sua provenance.
2. Con un bundle **non approvato** l'ammissione rifiuta la run.
3. Con lo stesso bundle e una **richiesta esplicita di root** la policy rifiuta il Pod.
4. Se il **gate** fallisce, la run si interrompe senza dichiarare l'immagine per la firma.

In nessun caso il team modifica la configurazione centrale o accede alla chiave di firma.

REEVO

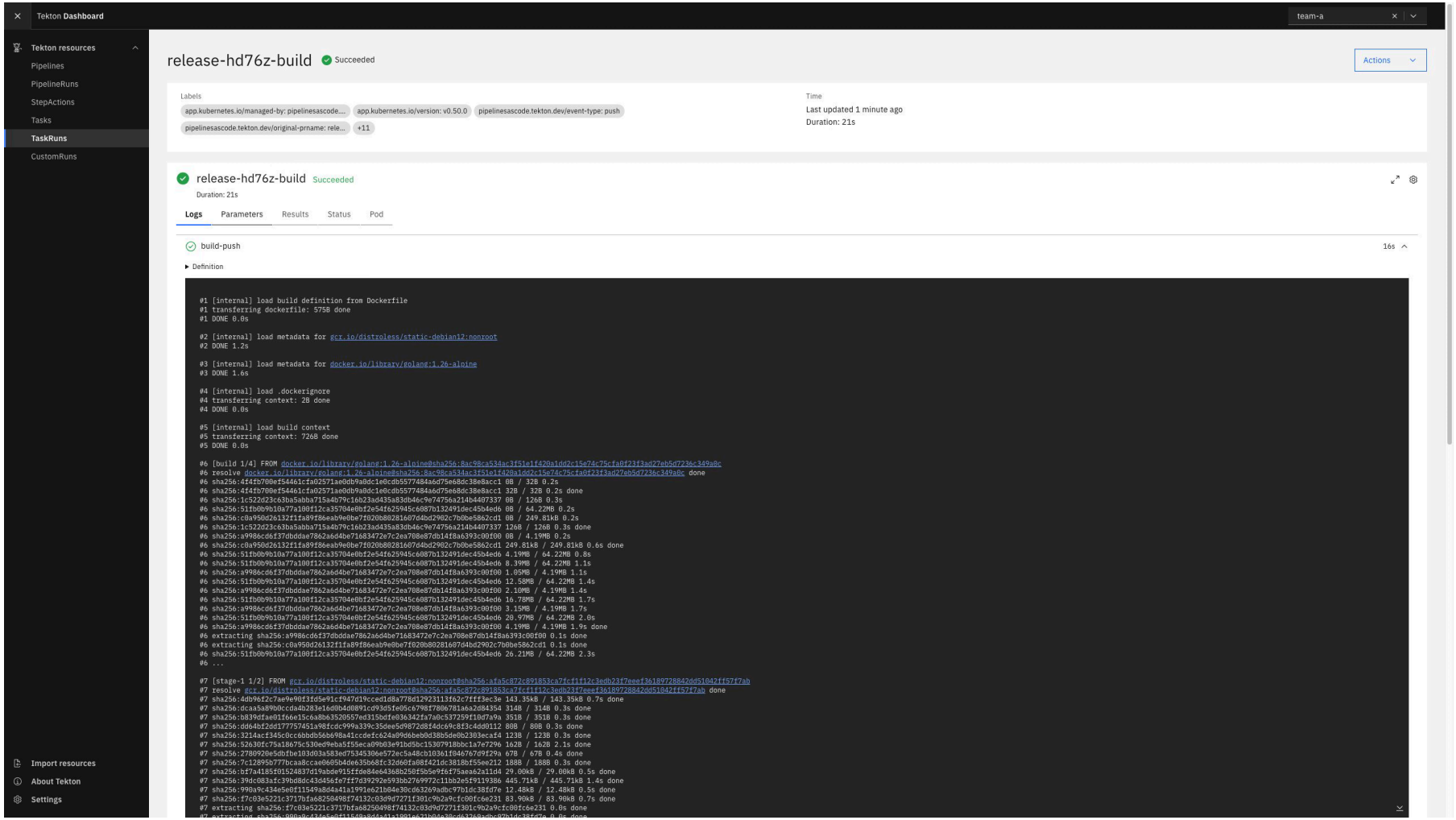
L'architettura della demo



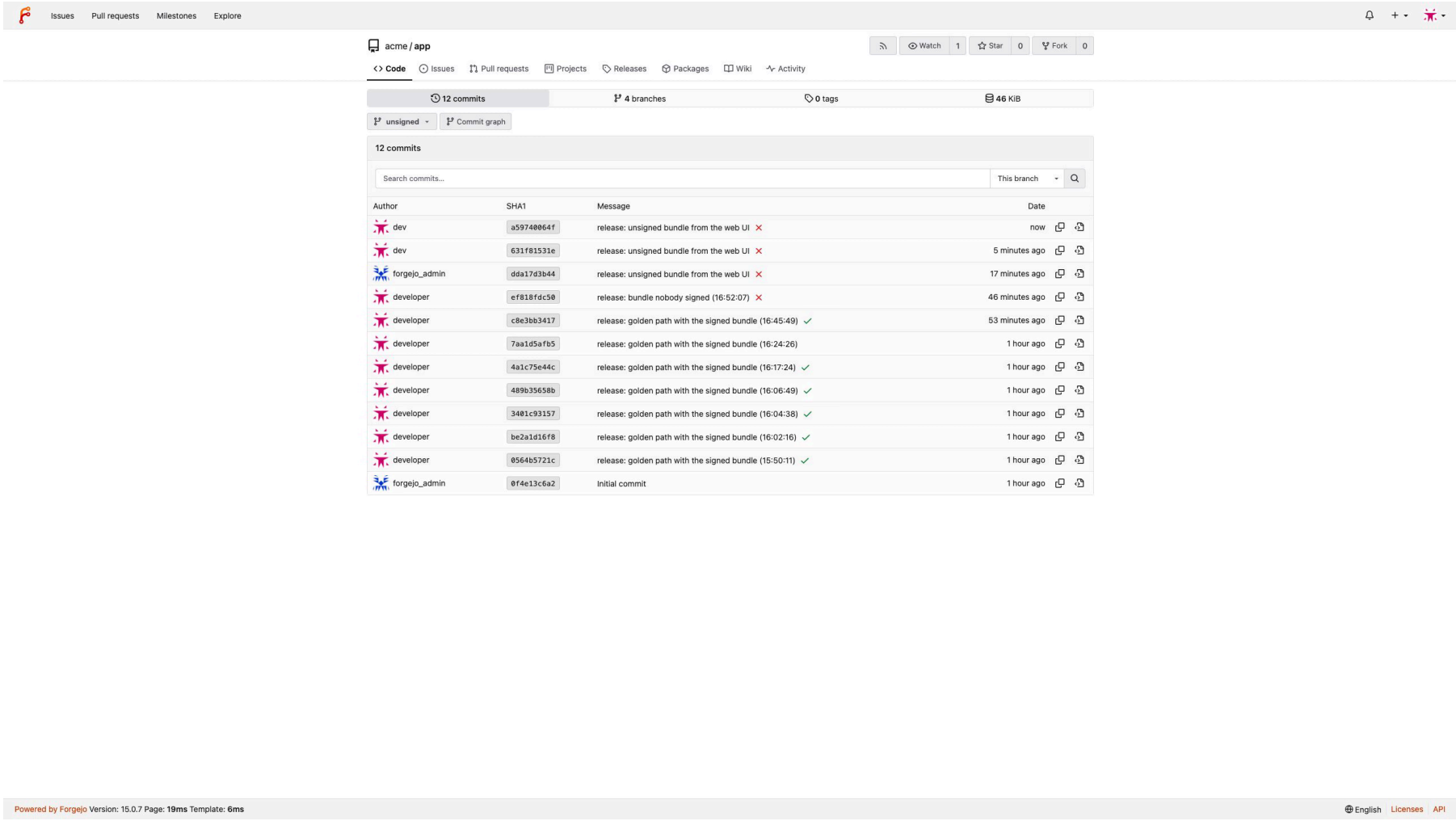
- Un push su Forgejo crea una run, che referenzia il bundle per digest.
- Ammissione e firma avvengono nel cluster; solo Chains ha accesso alla chiave.
- Il Pod di build esegue codice non fidato; i controller sono il confine fidato.

Diagramma: assets/demo-architecture.mmd

Esempio 1 · il push, la build, la firma

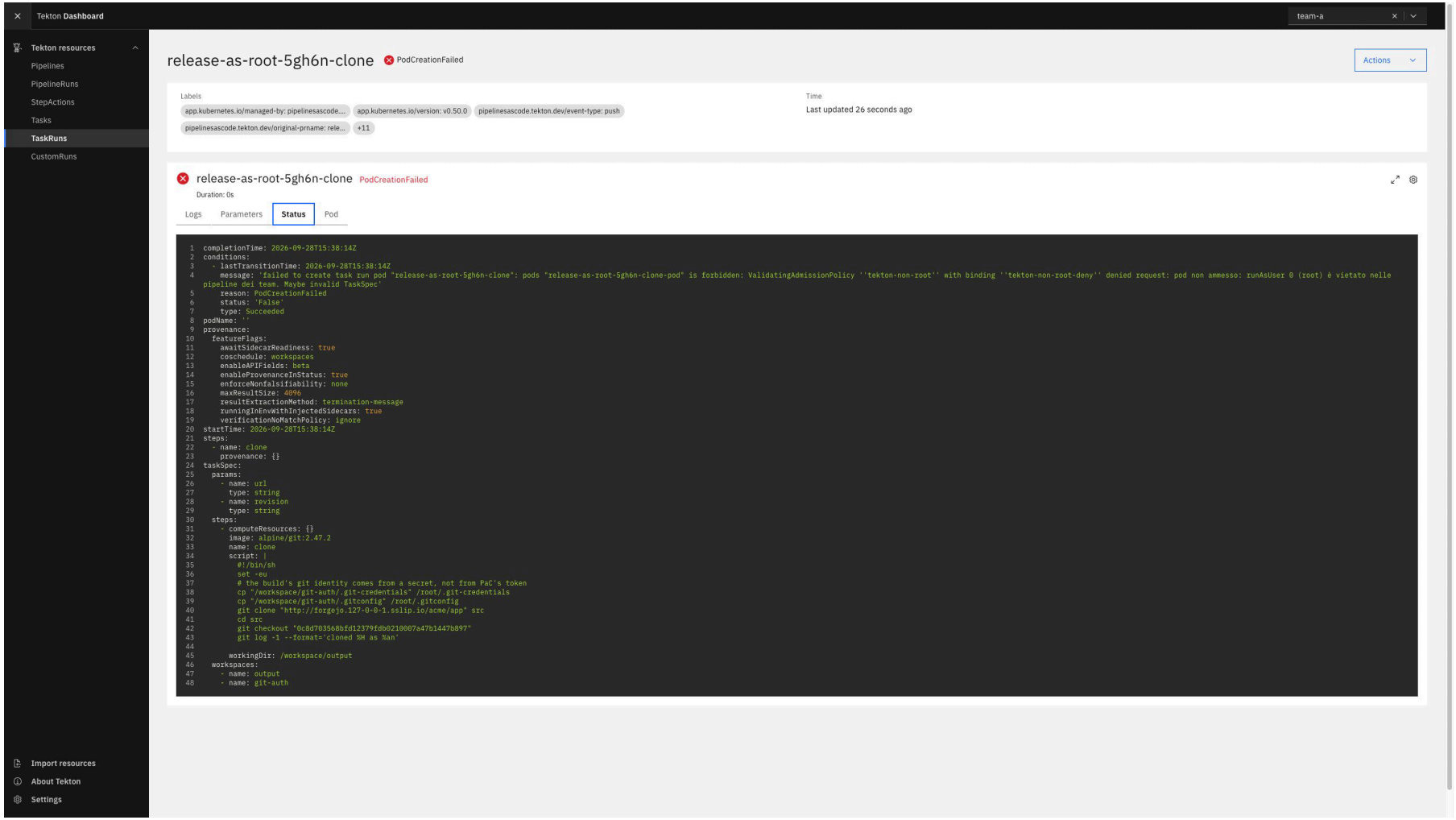


Esempio 2 · il bundle non approvato



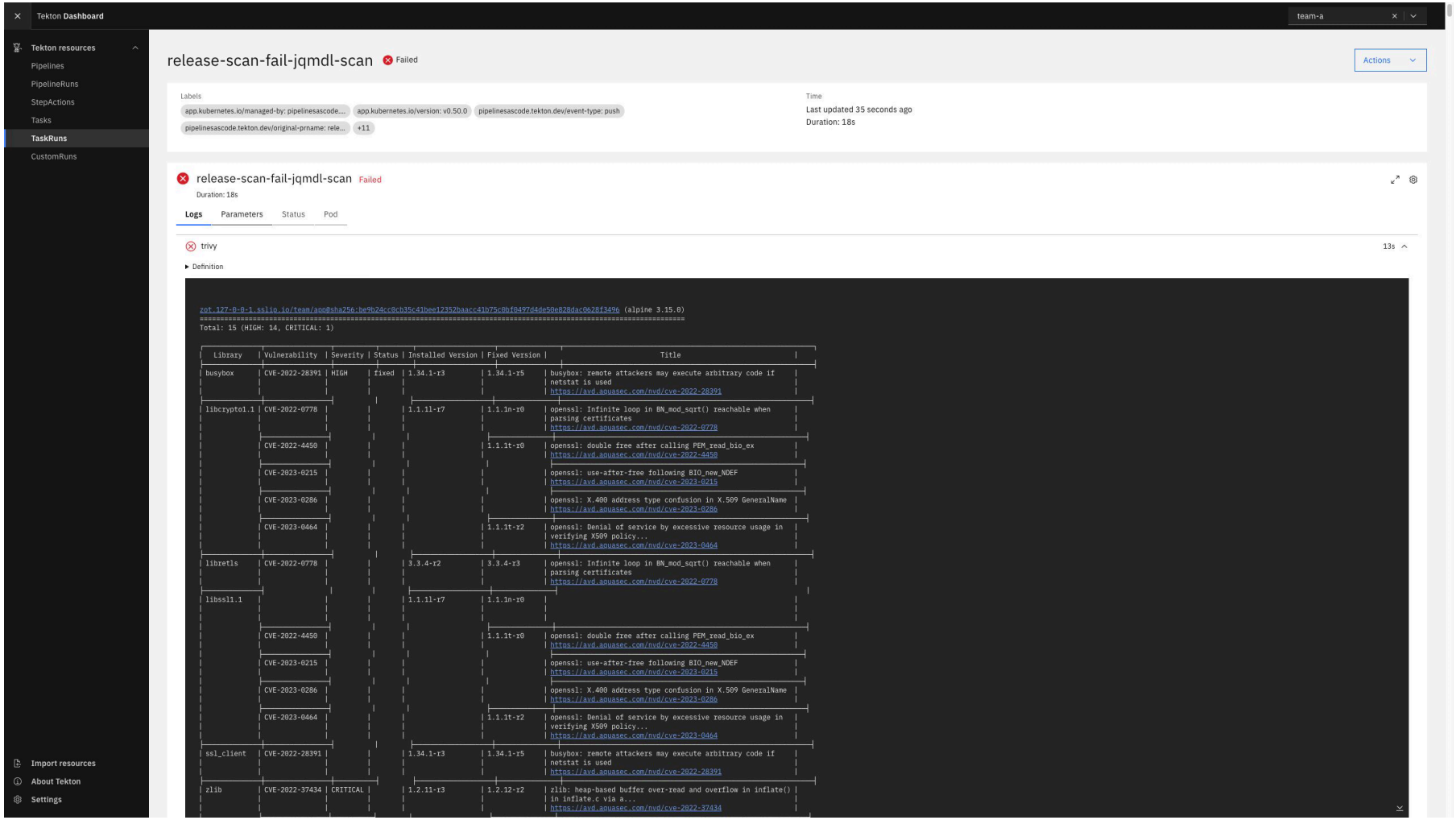
Push su unsigned → il commit segna ✗ → nessuna run nasce: l'ammissione ha rifiutato il digest

Esempio 3 · avviato come root



Push su as-root → la run nasce e fallisce: la policy rifiuta il Pod che chiede uid 0

Esempio 4 · il gate fallisce



REEVO

Takeaway

- **Sovranità**: controllo su esecuzione, dati, chiavi e possibilità di migrazione.
- **Piattaforma**: template condivisi, team autonomi e confini imposti centralmente.
- **Fiducia**: provenance firmata e verificata al consumo.

Grazie!

Domande?

Marco Russo · ReeVo Cloud & Cyber Security
marco.russo@reevo.it · GitHub · LinkedIn

Repo Github

Agenda della conferenza

REEVO

Fonti (1/2)

Attacchi

CISA — tj-actions (CVE-2025-30066) · GHSA-mw4p-6x4p-x5m5 — 23.000+ repository

CISA — Shai-Hulud, npm · CISA — SolarWinds Orion · Codecov — post-mortem 2021

Aqua — Trivy (CVE-2026-33634) · Microsoft — caso Trivy

Verizon — DBIR 2026

Lock-in e sovranità

U.S. DOJ — CLOUD Act · Trump v. Slaughter (25-332)

Dir. UE 2022/2555 — NIS2 · Reg. UE 2022/2554 — DORA · Reg. UE 2024/2847 — CRA

GitLab Docs — migrare da GitHub Actions («not exact matches») · GitHub Docs — tipi di custom action

Travis CI — fine del piano open source illimitato (2020) · curl/curl#7150 · CircleCI — sunsetting config 2.0

Fonti (2/2)

REEVO

Standard e offerta gestita

SLSA — Build: Track Basics (v1.2) · Build requirements

GitHub — Artifact Attestations · SLSA Build Level 3

GitLab — SLSA Level 3 provenance · firma self-hosted

Tekton

BuildKit — rootless mode · Tekton — Workspaces

Tekton — Concept model · Trusted Resources

Tekton — Chains · SLSA provenance · configurazione

Pipelines-as-Code · provider supportati

CNCF — Tekton incubating · tektoncd/community — adopters.md

tektoncd/cli#2214 · #2534 · #2894 · #2895

Ecosistema

Konflux · architettura: chiave long-lived, keyless come direzione · Kueue · Backstage (CNCF)

Backup · Le cinque dimensioni della sovranità

La **sovranità** indica quale organizzazione controlla la catena che produce il software: chi legge il codice, chi firma gli artefatti e chi può essere obbligato a consegnarli.

Dimensione	La domanda
Residency	Dove stanno i dati?
Giurisdizione	A quale ordinamento risponde il fornitore?
Chiavi	Chi le genera, chi le custodisce, chi può revocarle?
Operations	Chi può entrare nel sistema mentre gira?
Exit	Quanto costa uscire, e con quali prove?

Le cinque dimensioni sono indipendenti e vanno valutate separatamente.

REEVO

Backup · Non si parte da zero

- **Konflux**: piattaforma open source basata su Tekton che dichiara SLSA L3. Garantisce l'integrità con Trusted Tasks e Trusted Artifacts; la sua documentazione indica come limite una chiave di firma long-lived nel cluster, con il keyless come evoluzione prevista.
- **Kueue** (Kubernetes SIG): code e quote tra team.
- **Backstage**: portale per sviluppatori e golden path.

Fonti: konflux-ci.dev; Kueue (kubernetes-sigs); Backstage (CNCF)

REEVO

Backup · Trusted Resources: i vincoli misurati

Trusted Resources funziona, ma richiede attenzione alle versioni. I vincoli seguenti sono emersi nei nostri test:

- La firma è calcolata come `sha256(json(oggetto))`, quindi la CLI e Tekton devono serializzare l'oggetto negli stessi byte.
- Da Kubernetes 1.34 `ObjectMeta.creationTimestamp` usa `omitzero` (k/k#130989), e le CLI più recenti non emettono più `creationTimestamp: null`. Il JSON cambia e la verifica può fallire: controllare condizioni e log del controller.
- Con Pipelines v1.6.0 la verifica funziona con `tkn` 0.40 e 0.41, ma non con 0.43 e 0.46, anche se 0.43 è la versione indicata per quella release.
- Nel percorso CLI testato la firma è limitata alle risorse v1beta1 (cli#2214, chiusa citando la deprecazione di `tkn chains`), e su questo percorso ci sono bug ancora aperti (#2894, #2895).

Fonti: k/k#130989; tektoncd/cli#2214, #2534, #2894, #2895